Day 7 - Autoencoders and Generative Models

Networks that compress - and networks that create

Annika Stiehl · Deep Learning · M.Sc. · SRH

Today

Time	
09:00–09:10	Report + poster in 5 minutes
09:10–11:40	**Project time**
11:40–12:15	**Poster draft round** – every team on the big screen
13:00–14:30	**Recap**: Transformer and transfer learning • **Autoencoders**: compress without labels • denoise • find abnormal heartbeats
14:45–15:45	**Generative models**: VAE • GAN • a look at diffusion
15:45–16:00	Wrap-up

💻 After lunch: `day7a_autoencoders` and `day7b_generative` in Colab – a **CPU runtime** is enough today.

Report + poster in 5 minutes

Report (IEEE template, 2-4 pages)

- **Introduction** - problem, why it matters
- **Data** - DermaMNIST, splits, class imbalance
- **Methods** - baseline (A) → your improvements (B)
- **Results** - one table: balanced accuracy on the **test** split
- **Discussion** - what worked, what did not, why
- **plus**: AI use statement, contribution statement

Poster (shown on the big screen on 11 Nov)

- **one message** in the title: what did you find?
- **one page** (PDF), landscape - shown full-screen
- one big results figure / table - readable from the back row
- more pictures, less text: ≈ 300 words
- QR code to your code / app

Dates: 10 Nov `model.pt` · 11 Nov poster · 15 Nov CampusWeb

🖼️ Today 11:40

Every team shows its **poster draft** (on screen, 3 min) - a sketch is enough.

Project time

09:10-11:40

Project time

- **free work** in your teams on the challenge
- finish your **model** (due 10 Nov!) - save `model.pt` and test that it loads
- start the **poster**: one page, title = your message, one big results figure
- questions? I come round to every team
- breaks: whenever your team likes

Dates

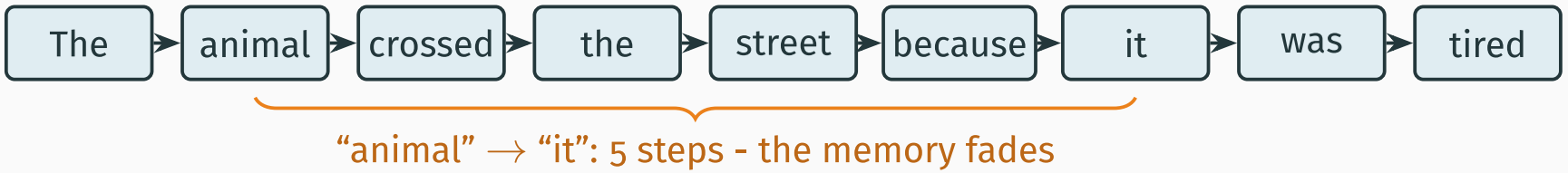
- **10 Nov**: send your model (`model.pt`)
- **11 Nov**: poster presentations on the big screen · leaderboard
- **15 Nov**: hand in on **CampusWeb**: code, model, report, poster, app

Recap

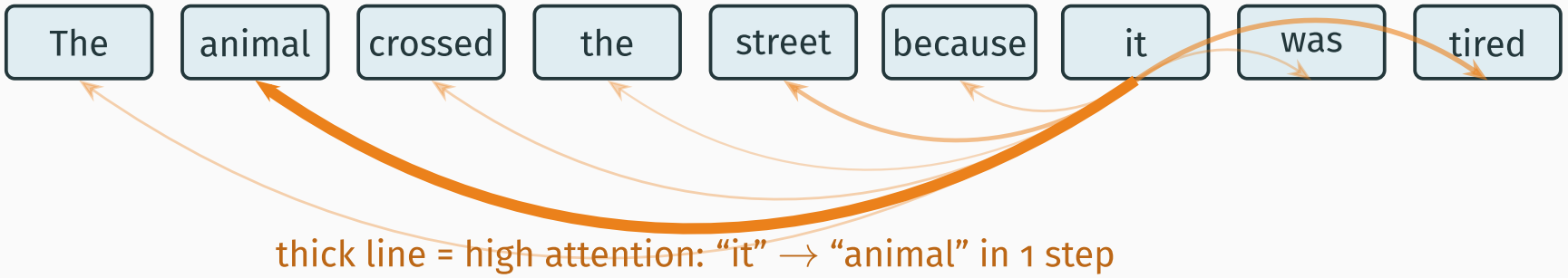
Transformers and transfer learning

Last time: the Transformer

RNN: one memory, passed on step by step



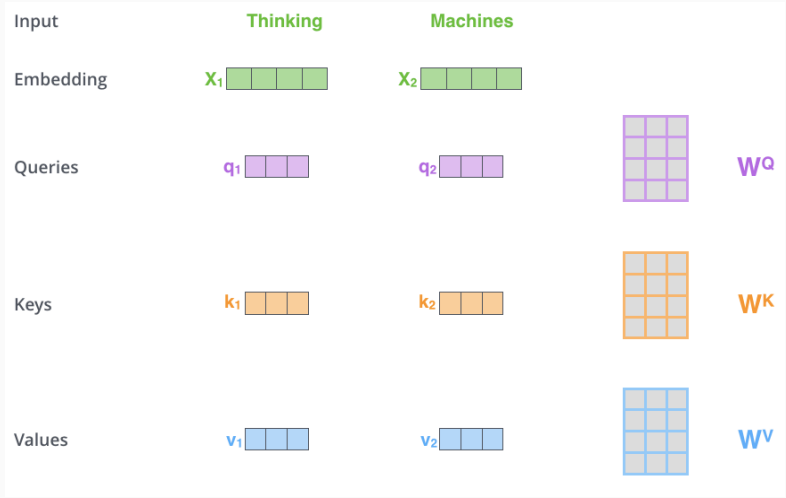
Attention: every word looks at every other word directly



- **attention**: every word looks at every other word directly - query, key, value; many heads
- **Transformer** = attention + MLP blocks, stacked; positions are added
- **pre-trained** on huge amounts of text or images - **without labels** (guess the hidden / next word)

Recap: query, key, value - and positional encoding

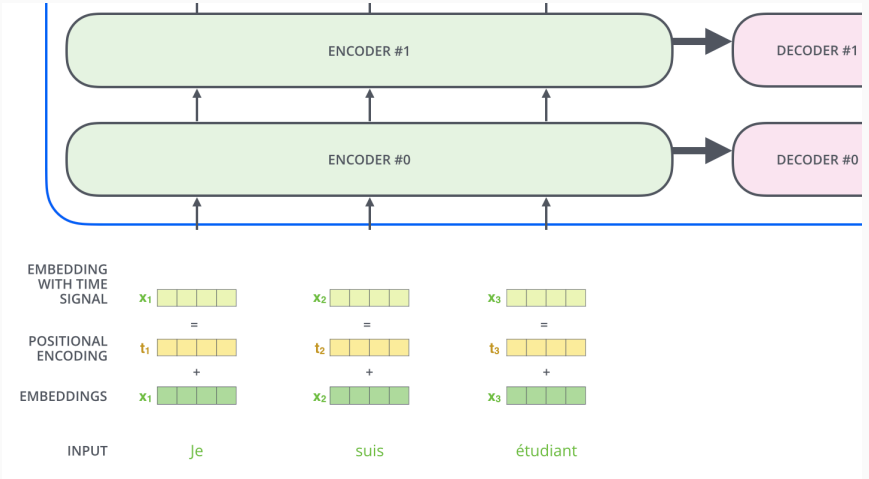
Query, key, value = a search (like a library)



query	what this word **looks for**
key	what each word **offers**
value	what each word **passes on**

query · key match → high weight (softmax) → mix of the **values**

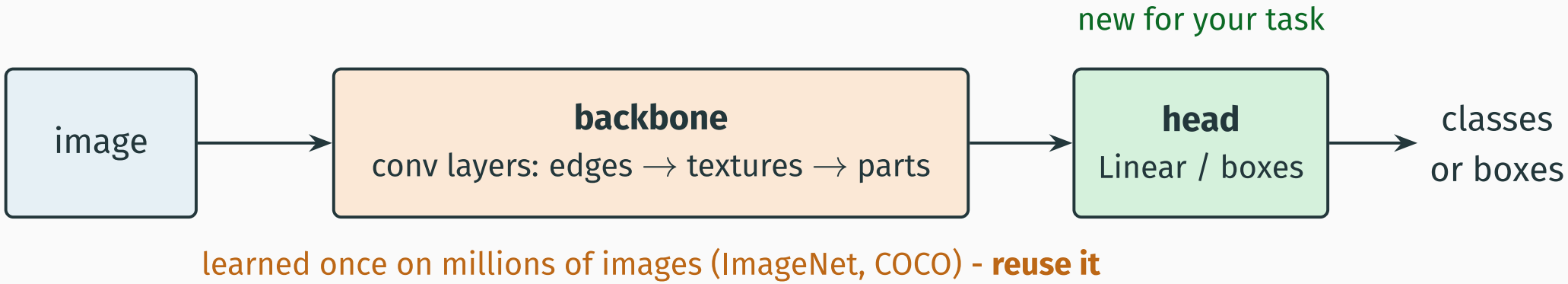
Positional encoding = where is the word?



- attention alone does **not know the order**: "dog bites man" = "man bites dog"
- fix: **add** a position vector to every word vector
- the RNN got the order for free (step by step)

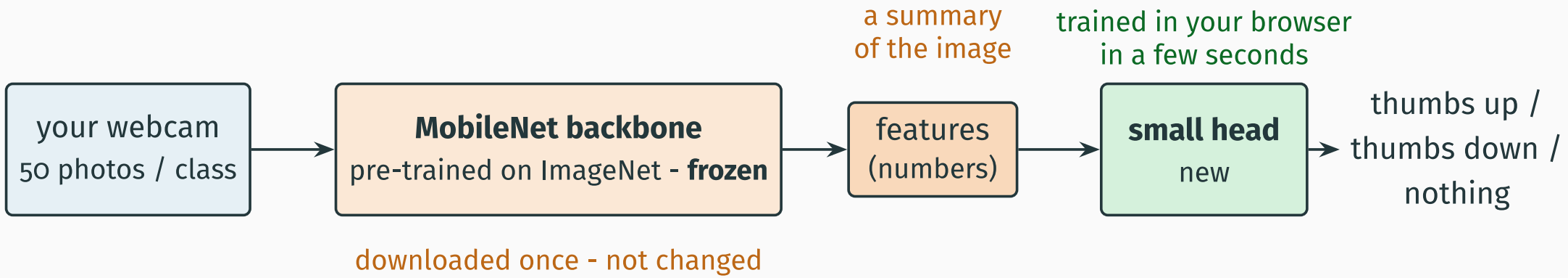
Figures: J. Alammar, The Illustrated Transformer, CC BY-NC-SA 4.0

Why transfer learning?



- labels are **expensive** (our brain scans: 893, labelled by doctors) - but pre-trained networks are free
- **feature extraction**: freeze the backbone, train only the new head - little data
- **fine-tuning**: train everything with a small learning rate - more data

Remember day 1? Teachable Machine was transfer learning



- on day 1 you trained a classifier with **50 webcam photos** in **seconds** - in a browser
- the trick: a **pre-trained MobileNet** (ImageNet) is frozen, only a small head learns
- Advanced: epochs 50, batch size 16, learning rate 0.001

Link

teachablemachine.withgoogle.com → Image project.

Autoencoders

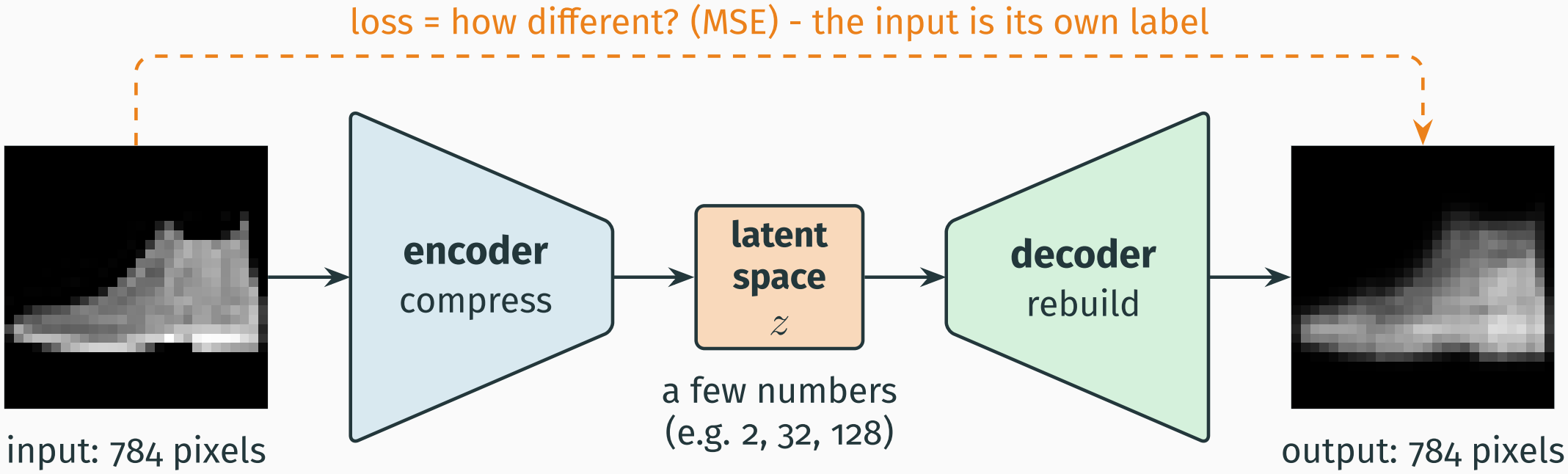
Compress without labels - the picture is its own label

Learning without labels

	what the network gets	example
supervised (days 1-6)	input **+ label**	skin photo + "melanoma"
unsupervised / self-supervised	**only the input**	10 000 hours of EEG, no annotations

- labels need experts and time - **raw data** is often cheap and plentiful
- idea: give the network a task that needs no labels - e.g. **copy your input**
- sounds trivial - unless the copy has to pass through a **narrow door**

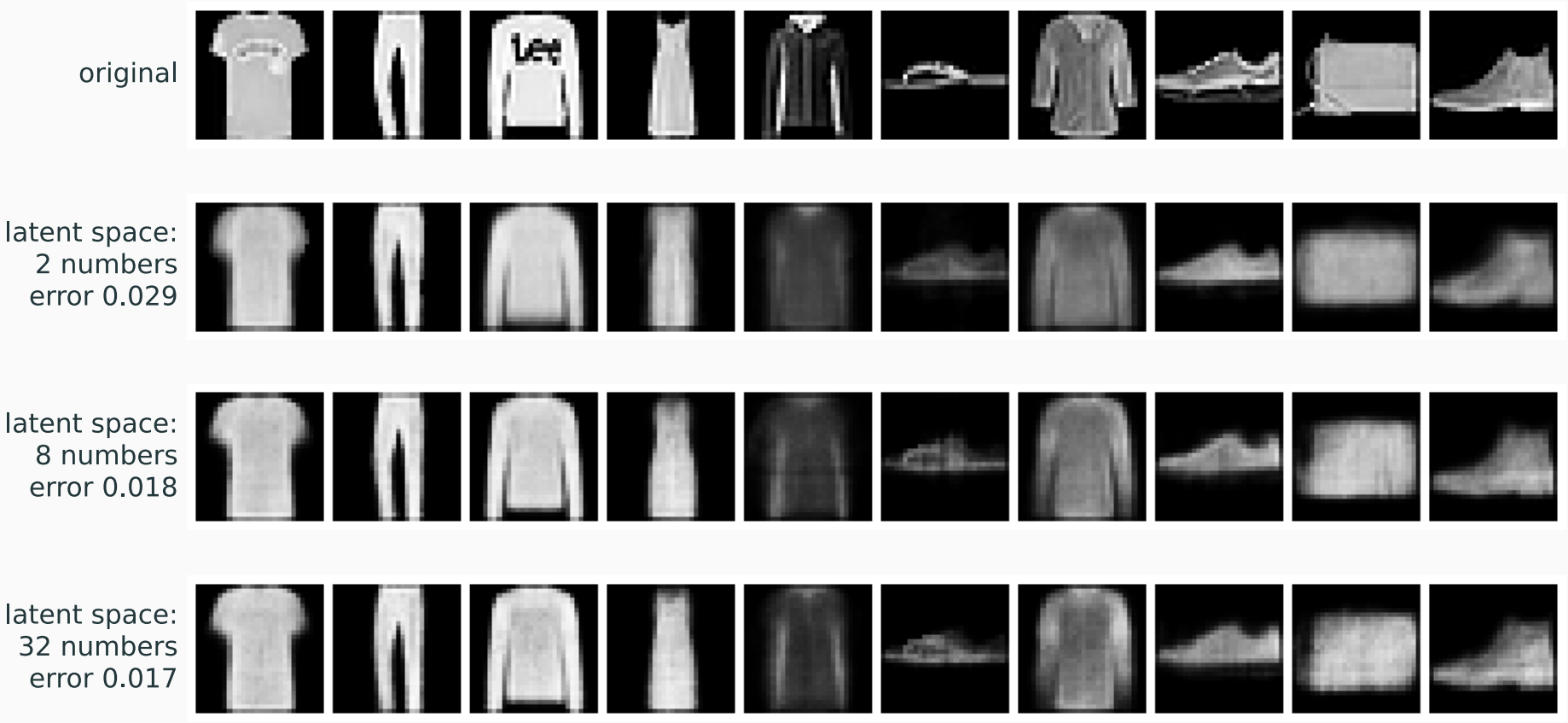
The autoencoder



bottleneck: the only way through

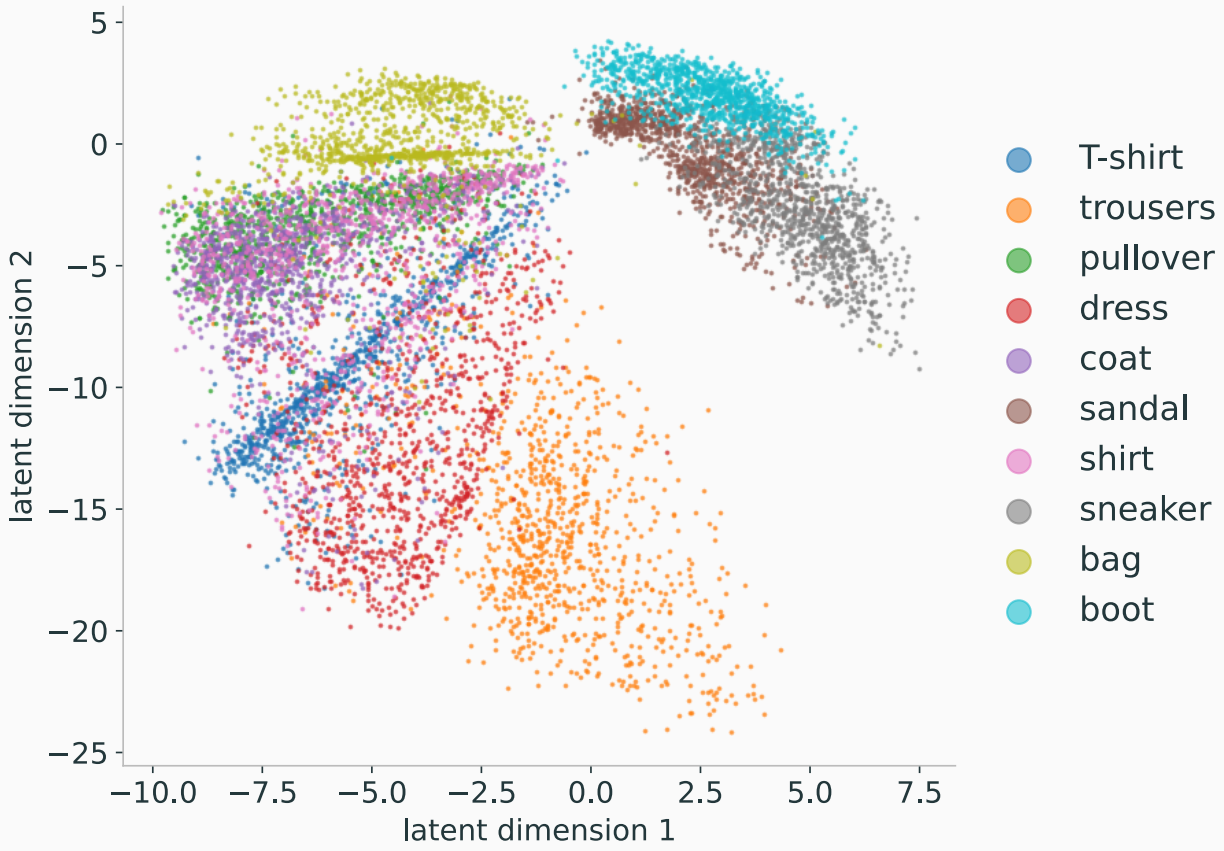
- **encoder**: 784 pixels → a few numbers in the **latent space** (one point there = a latent vector, also called embedding) - how many is your choice: 2, 32, 128 ...
- **decoder**: latent vector → 784 pixels again
- **loss**: MSE between input and output - **no label needed**; the network must keep only what matters

How small can the bottleneck be?



FashionMNIST, 5 epochs: **2 numbers** → only the rough shape, a shirt becomes a pullover · **32 numbers** → good, but **blurry** - fine details are lost first

The latent space is a map



- latent space with **2 numbers** → every picture is a **point**
- colours = classes - the network **never saw** them
- similar clothes land **close together**: shoes next to each other, tops overlap
- the autoencoder **sorted the data by itself**

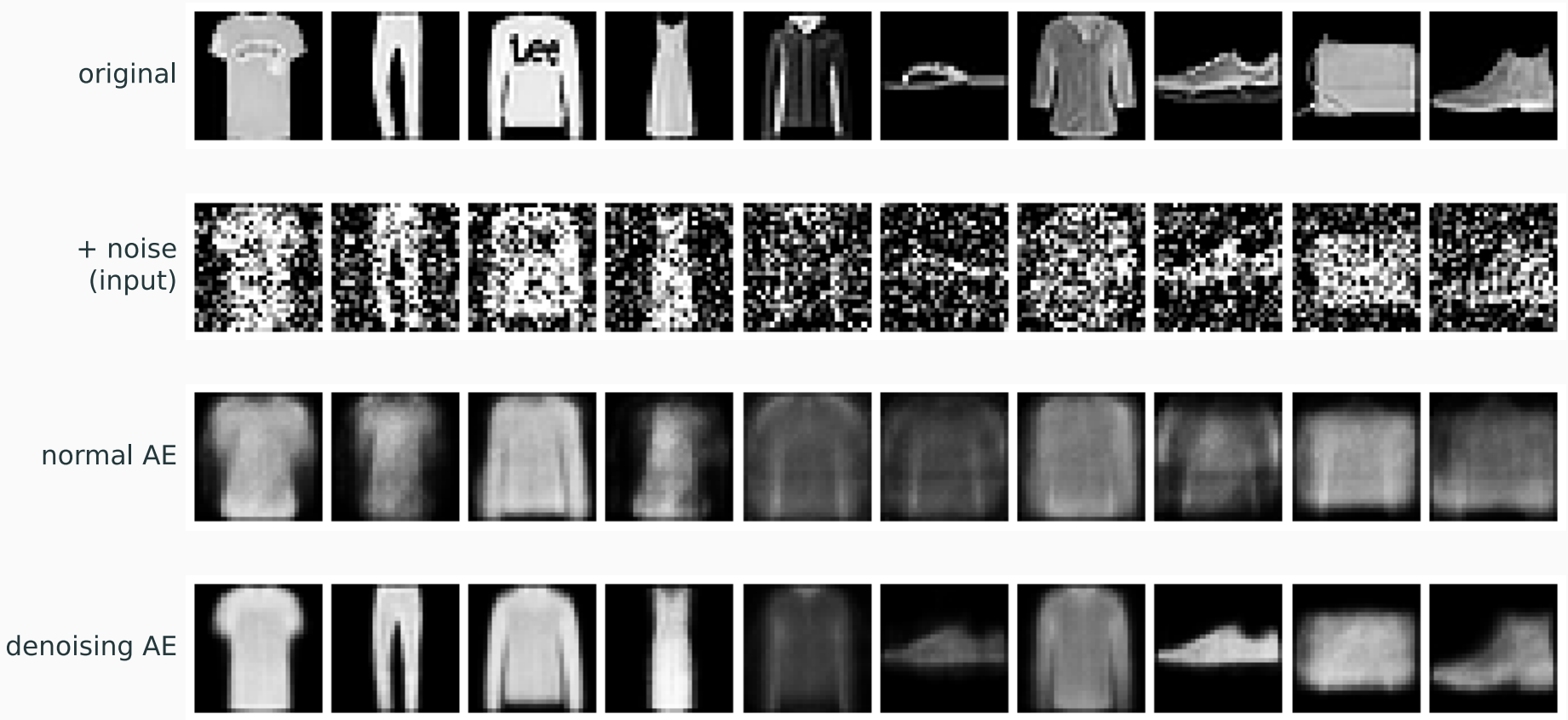
A linear autoencoder = PCA

📖 Deep dive · self-study

- no activation functions + MSE loss → the autoencoder learns the same **subspace** as **principal component analysis** (PCA)
- PCA: rotate the data so that the first axis has the **largest variance**, the second the next largest, … keep the first k axes
- encoder $z = Wx$, decoder $\hat{x} = W^\top z$ - the best W (for MSE) spans the first k principal components
- with **ReLU** layers the autoencoder can **bend** the map → classes separate better than with PCA (try: `sklearn.decomposition.PCA(2)` on the same data and compare the two maps)
- PCA has a closed-form solution (no training) - always a good **baseline** for an autoencoder

Baldi & Hornik 1989; Hinton & Salakhutdinov, Science 2006

The denoising autoencoder



- training: **input** = picture + noise, **target** = the clean picture → the network learns what clothes look like
- error on the test set: noisy **0.118** → normal AE **0.056** → denoising AE **0.023**

Cleaning EEG with denoising networks

📖 Deep dive · self-study

- EEG is full of **artefacts**: eye blinks and eye movements (EOG), muscle activity (EMG), electrode pops, line noise
- classical: filters, ICA (independent component analysis) - often needs a human to choose components
- learned: train on pairs **(clean EEG + artefact, clean EEG)** - exactly the denoising autoencoder
- **EEGdenoiseNet** (Zhang et al., J. Neural Eng. 2021): a public benchmark with 4 514 clean EEG, 3 400 EOG and 5 598 EMG segments - mix them at a chosen SNR and train a network to remove the artefact
- pitfall: a network trained on **simulated** mixtures can fail on **real** artefacts - always check on real recordings
- other denoising models in medicine: low-dose CT, fast MRI (fewer measurements, then a network fills in the details)

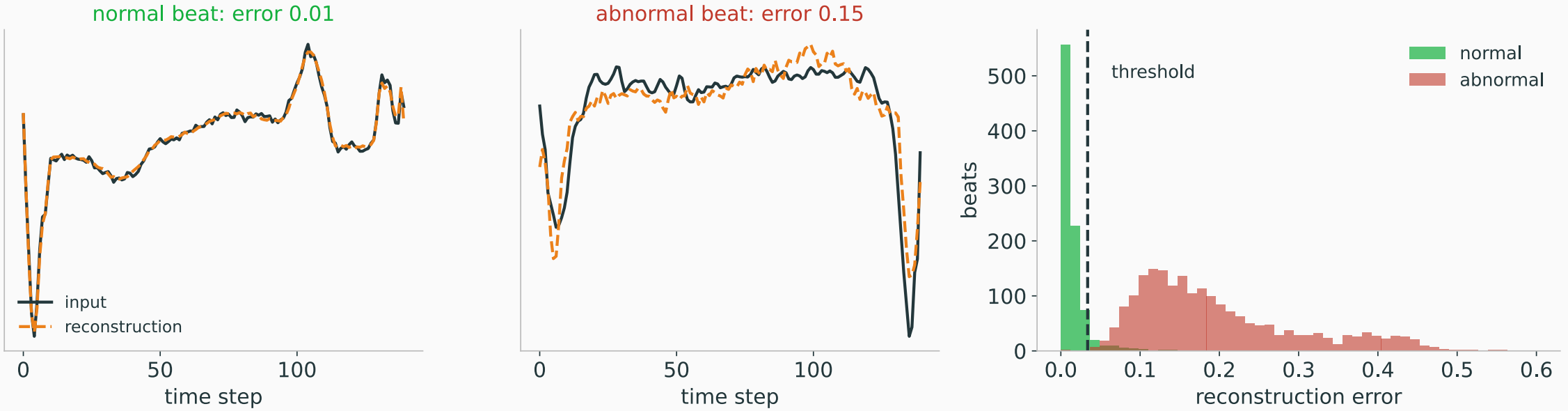
What are autoencoders used for?

- **denoising**: EEG artefacts, low-dose CT, old photos
- **compression**: a few numbers in the latent space instead of the full signal
- **features without labels**: train the encoder on unlabelled data, then put a small head on it (= transfer learning)
- **anomaly detection**: train only on **normal** data - something unusual is rebuilt badly → **large error = alarm**
- **generation**: the decoder paints new pictures from new points in the latent space? → block 2

🧠 EEG corner

Seizures are rare and every patient is different - but **normal** EEG is plentiful. Train an autoencoder on normal EEG, raise an alarm when the error is large.

Example: finding abnormal heartbeats (ECG5000, day 5)



- trained **only on 2 000 normal beats**, no labels for abnormal beats
- threshold = 95 % of the normal training beats are below → new normal beats: **8 %** false alarms · abnormal beats: **100 %** found (AUC 0.99)

Anomaly detection: pitfalls

📖 Deep dive · self-study

- **the threshold** decides everything: lower → more alarms found, more false alarms (sensitivity vs specificity, day 3) - choose it on **validation** data, not on the test set
- **"normal" must really be normal**: a few hidden anomalies in the training data and the network learns to rebuild them too
- a strong decoder can rebuild **everything** well - also anomalies → keep the bottleneck small, check that abnormal errors are really larger
- the error measures **"unusual"**, not **"sick"**: artefacts, a loose electrode or a new patient are unusual too
- report it like a classifier: AUC, sensitivity at a fixed false-alarm rate, false alarms **per hour** (for EEG)
- the 8 % / 100 % on the last slide: one dataset, one seed, one patient - easy

Autoencoders in one slide

autoencoder	encoder → small latent space → decoder; loss = rebuild error (MSE) - **no labels**
bottleneck	small → only the rough shape; larger → more details
latent space	a map: similar inputs land close together
denoising AE	noisy input, clean target → learns what the data looks like
anomaly detection	train on normal data only → large error = unusual

❓ Think about it

The decoder turns a point in the latent space into a picture. What happens if we give it a point that **no picture** had - a random point on the map?

Generative models

Networks that create new data - VAE, GAN, diffusion

Which face is real?

whichfaceisreal.com - two faces, one is a photo, one was made by a network (StyleGAN)

- play 5 rounds - how many do you get right?
- what gives the fake away? (look at backgrounds, ears, glasses, hair)

thispersondoesnotexist.com - a new face every time you reload

👥 Try it (3 min)

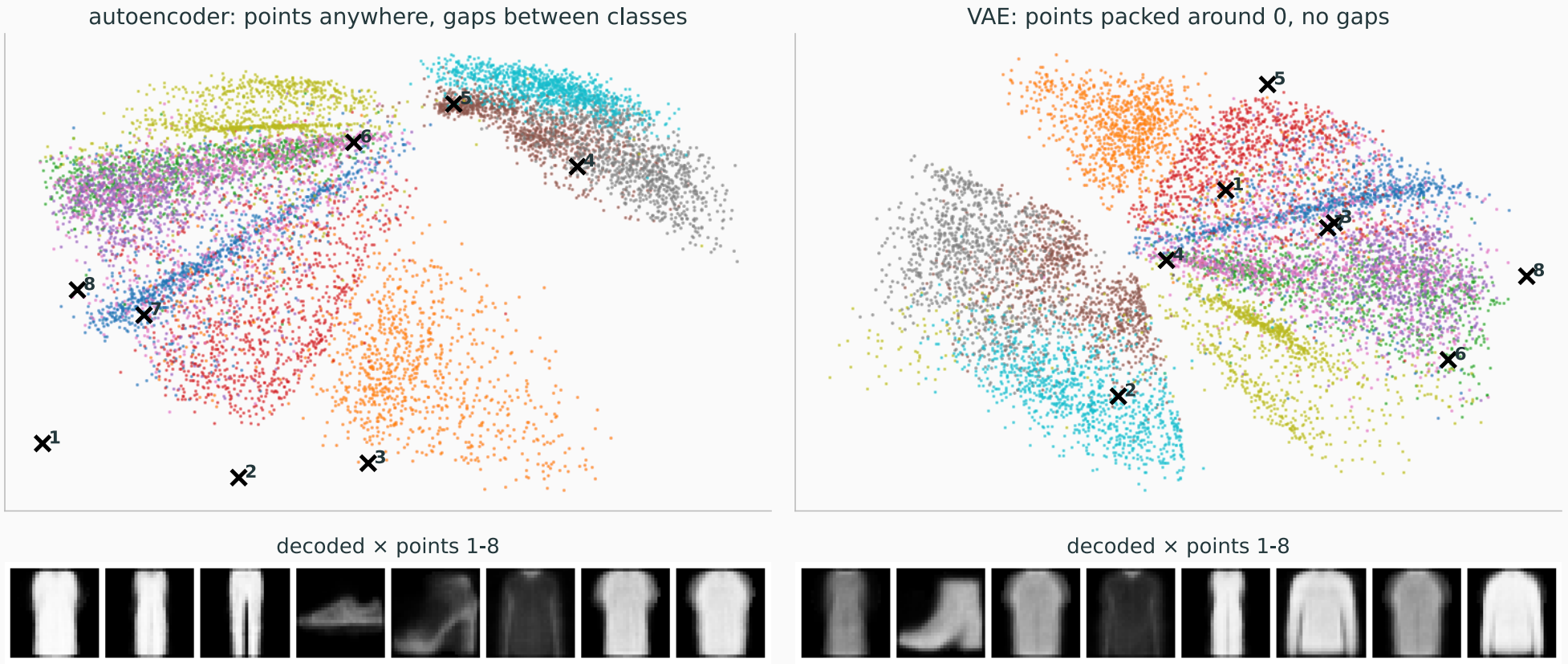
With your neighbour. Count your score.

Recognise vs create

	discriminative (days 1-6)	**generative (today)**
question	"is this a melanoma?"	"what does a melanoma look like?"
learns	the **border** between classes	the **data itself**
output	a label	**new** examples
examples	ResNet, YOLO, BERT	VAE, GAN, Stable Diffusion, ChatGPT

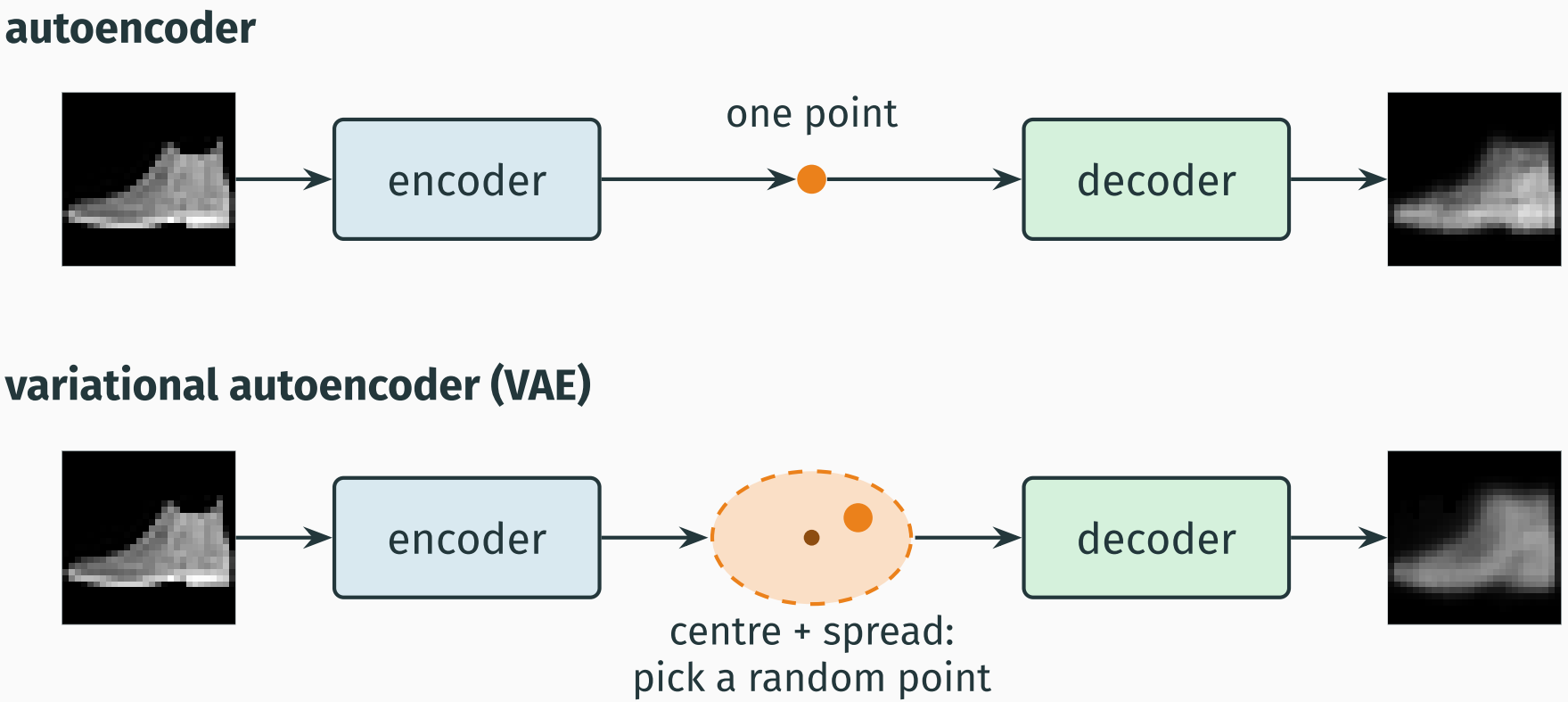
To create, a network must learn which pictures are **likely** - and then pick a new one.

Random points in the latent space: autoencoder vs VAE



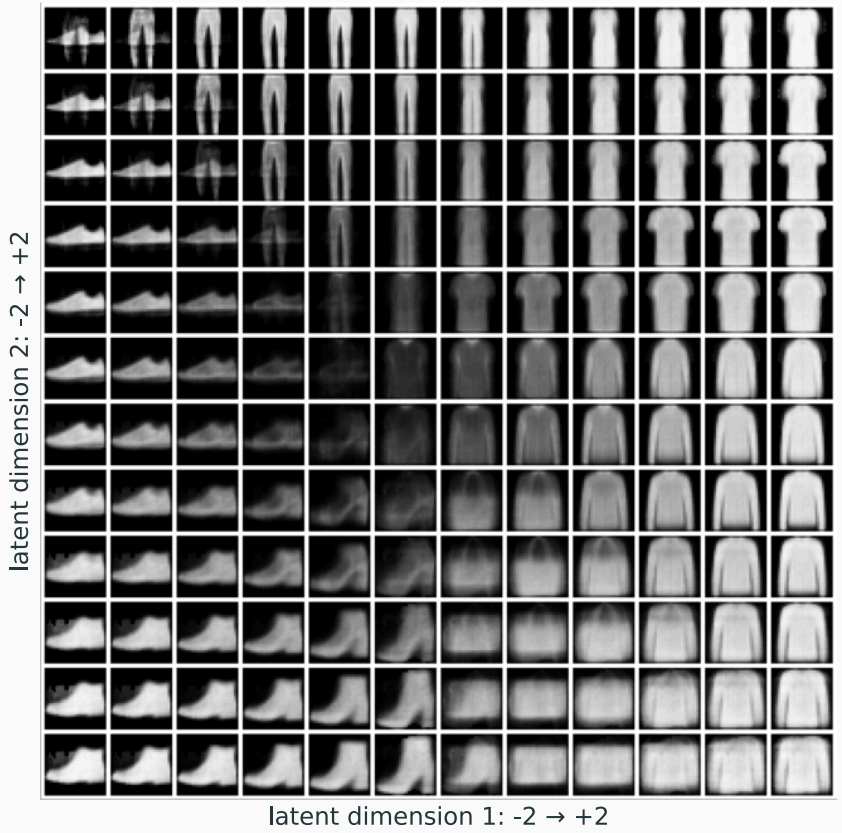
- **autoencoder** (left): odd shapes and **gaps** - where should we pick? Points in a gap or outside give dark or mixed pictures
- **VAE** (right): the latent space is packed around 0 → simply pick from a **bell curve** → mostly new clothes

The VAE: a cloud instead of a point



1. the encoder gives a **centre** and a **spread** - a small cloud; the decoder gets a **random point** from it
2. the loss has two parts: **rebuild** the picture + keep every cloud **close to 0** with spread ≈ 1
3. → clouds overlap, the gaps close, every point near 0 means something

A walk over the VAE map



- decode a **regular grid** of points in the latent space from -2 to +2
- one kind of clothing **morphs smoothly** into the next: trousers → T-shirts → pullovers, sneakers → boots → bags
- every picture here is **new** - none of them is in the dataset
- typical for a VAE: a bit **blurry**

The VAE loss (ELBO) and the reparameterisation trick

📖 Deep dive · self-study

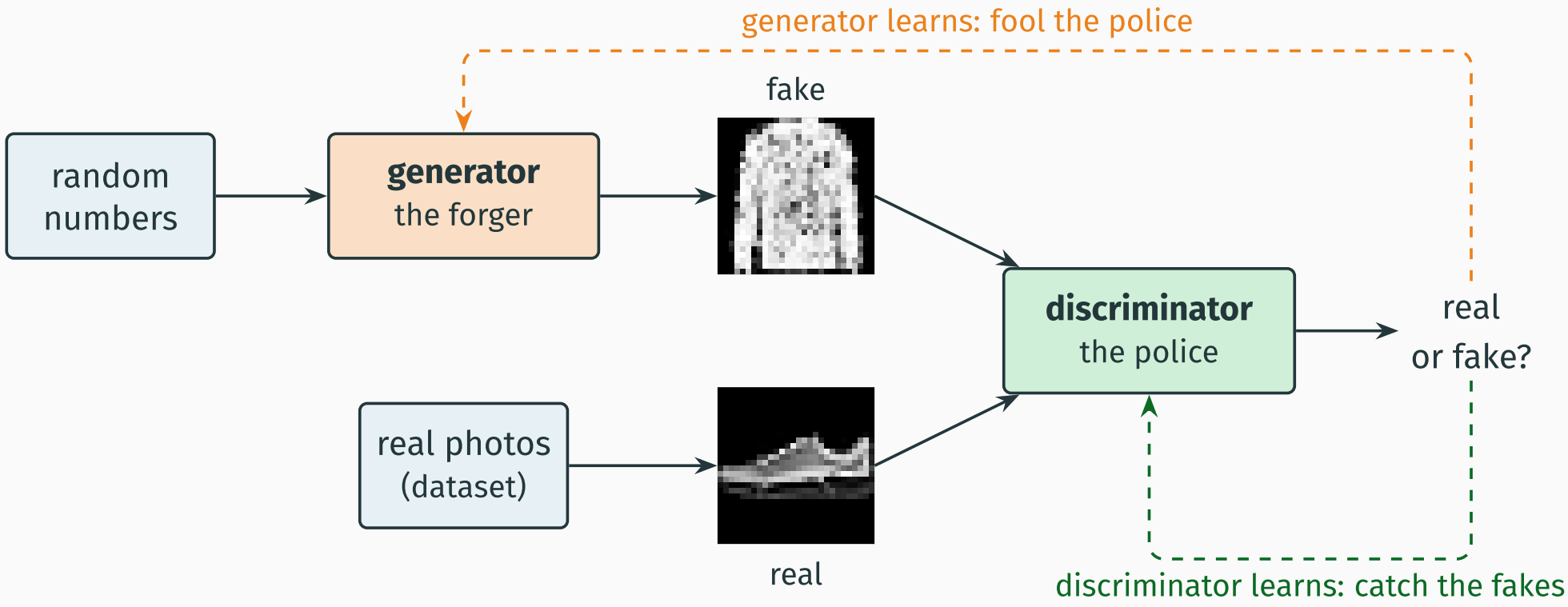
- encoder: $q(z \mid x) = \mathcal{N}(\mu(x), \sigma^2(x))$ - a cloud; prior $p(z) = \mathcal{N}(0, 1)$ - where all clouds should be
- loss for one picture = **rebuild error** + **KL divergence** (how different is the cloud from $\mathcal{N}(0,1)$):

$$\mathcal{L}(x) = \underbrace{\|x - \mathrm{dec}(z)\|^2}_{\text{rebuild}} + \beta \cdot \underbrace{\tfrac{1}{2} \sum_j \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1\right)}_{\text{KL: stay close to } \mathcal{N}(0,1)}$$

- KL = 0 exactly when $\mu = 0$ and $\sigma = 1$ (try it: $0 + 1 - 0 - 1 = 0$)
- **reparameterisation trick**: we cannot backpropagate through "pick a random number" → write $z = \mu + \sigma \cdot \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 1)$: the randomness is an input, gradients flow to μ and σ
- the encoder outputs $\log \sigma^2$ (any real number) instead of σ (must be > 0) - in the notebook: `log_spread`
- minus this loss is (up to constants) a lower bound of $\log p(x)$ - the **evidence lower bound (ELBO)**; β = `KL_WEIGHT` (β-VAE)

Kingma & Welling, Auto-Encoding Variational Bayes, ICLR 2014

The GAN: forger against police



- **generator**: random numbers → a picture - it never sees a real picture
- **discriminator**: real or fake? - a normal classifier (day 2)
- they train **in turns** - each one makes the task harder for the other

Video: how a GAN learns

🎬 **IBM Technology: What are GANs (Generative Adversarial Networks)?** - Martin Keen, lightboard

mediacenter.ibm.com/media/What+are+GANs... (also on YouTube: IBM Technology - What are GANs)

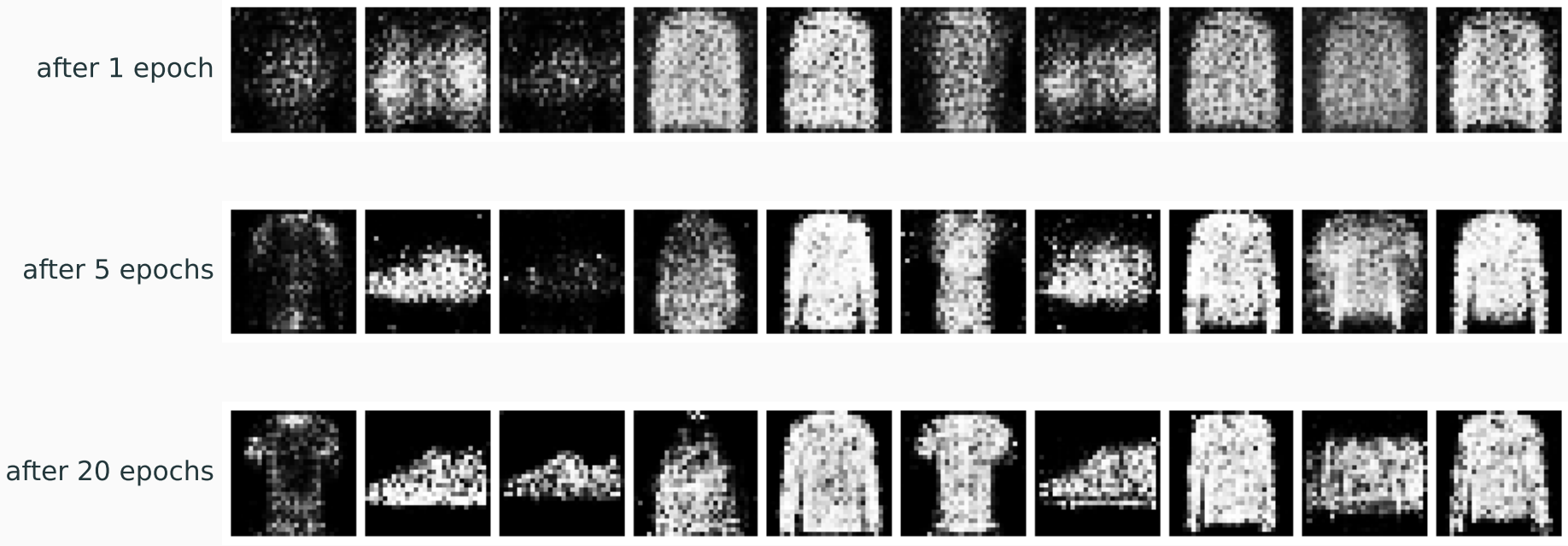
Watch for:

1. what does the **generator** get as input - and what does it produce?
2. how does the **discriminator** learn - and how does the generator learn from it?
3. when is the training "finished"?

🏠 Try it at home: GAN Lab

poloclub.github.io/ganlab - watch a GAN learn 2-D points in your browser. Choose the **ring**, press ▶, then try **two circles**: are both covered?

Our GAN on FashionMNIST



The same 10 random inputs after 1, 5 and 20 epochs (2 small MLPs, CPU, 20 epochs ≈ 3 min): hard edges and texture - but **noisy**, some pictures are broken. Real GANs (StyleGAN: convolutions, days on GPUs) make photo-realistic faces.

Why GANs are hard to train

- **unstable**: two networks chase each other - the losses go up and down, they do not tell you how good the pictures are
- **mode collapse**: the generator finds a few pictures that fool the discriminator - and only makes those (only sneakers, no bags)
- **one network too strong**: the discriminator wins easily → the generator gets no useful gradient
- **evaluation**: no single loss to read - people use FID (compares features of real and fake pictures) and look at samples

The GAN game in formulas - and how it is trained in practice

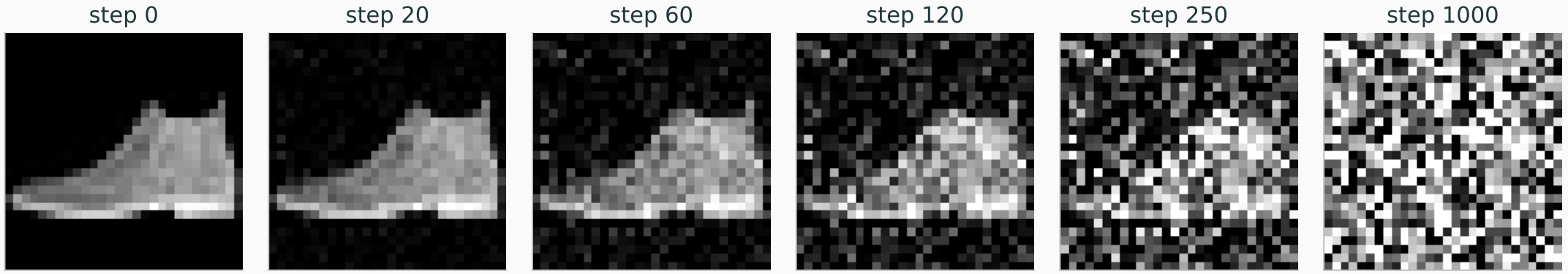
📖 Deep dive · self-study

$$\min_G \max_D \; \mathbb{E}_{x\sim\text{data}}\big[\log D(x)\big] + \mathbb{E}_{z\sim\mathcal{N}(0,1)}\big[\log(1 - D(G(z)))\big]$$

- $D(x)$ = probability "real"; the police maximises (real → 1, fake → 0), the forger minimises
- optimum: $D = 0.5$ everywhere and the fakes have exactly the distribution of the data
- **non-saturating loss** (what everybody uses, also our notebook): the forger maximises $\log D(G(z))$ instead - same goal, but a strong gradient at the start when the fakes are bad
- in PyTorch = binary cross-entropy (day 2): police `bce(D(real), 1) + bce(D(fake.detach()), 0)`, forger `bce(D(fake), 1)`
- **DCGAN** rules (Radford et al. 2016): convolutions instead of pooling, BatchNorm, LeakyReLU in D, Adam with lr 2e-4, $\beta_1 = 0.5$
- famous successors: **StyleGAN** (faces, Karras et al. 2019), **CycleGAN** (MRI ↔ CT style, horse ↔ zebra), **pix2pix**

Goodfellow et al., Generative Adversarial Nets, NeurIPS 2014

Outlook: diffusion models



→ forward: add a little noise, 1000 times (no learning) ← backward: a network removes the noise, step by step

- forward: add noise in many small steps until only noise is left (no learning)
- backward: a network learns to **remove a little noise** at each step - a denoising autoencoder, used 1000 times
- **Stable Diffusion, DALL·E, Midjourney** - and text tells the network what to paint · 👉 Diffusion Explainer

Diffusion models in a bit more detail

📖 Deep dive · self-study

- noisy picture at step t: $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon$ - a mix of the clean picture and noise; $\bar{\alpha}_t$ goes from 1 (clean) to ≈ 0 (pure noise)
- training (DDPM, Ho et al. 2020): pick a picture, a random step t and noise ε → the network (a U-Net) predicts ε from x_t and t → loss = MSE between true and predicted noise
- generating: start at pure noise, subtract a little predicted noise, repeat 1000 times (newer samplers: 20-50 steps)
- **latent diffusion** (Stable Diffusion): first compress the image with an **autoencoder**, run the diffusion in its small latent space - much faster
- **text-to-image**: the text goes through a Transformer, the U-Net looks at it with **attention** (day 6)
- in medicine: synthetic MRI / CT / skin images, image-to-image (MRI → CT), reconstruction of fast MRI

Ho, Jain & Abbeel, Denoising Diffusion Probabilistic Models, NeurIPS 2020; Rombach et al., CVPR 2022

Four ways to create

	autoencoder	VAE	GAN	diffusion
idea	compress + rebuild	cloud instead of point	forger vs police	remove noise step by step
new samples?	poor (gaps)	yes	yes	yes
quality	-	blurry	**sharp**	**sharp + varied**
training	easy	easy	**unstable**	easy, but slow
generating	-	fast (1 step)	fast (1 step)	slow (many steps)
known from	denoising, anomalies	drug molecules	deepfakes, StyleGAN	Stable Diffusion, DALL·E

Generative models in medicine

Chances

- **more data for rare classes**: synthetic melanomas, synthetic seizure EEG
- **privacy**: share synthetic data instead of patient data
- **image-to-image**: MRI → CT, fast MRI, denoising low-dose CT

Risks

- a model can **invent** a lesion - or remove one
- synthetic data can copy (= leak) real patients
- **deepfakes**: fake X-rays, fake documents
- tested on synthetic data ≠ works on real patients

❓ Think about it

Your challenge: melanoma is a rare class. Would you add GAN-generated melanomas to your **training** data? To your **test** data?

Cheat sheet: generative models in one slide

generative model	learns what the data looks like → creates **new** examples
VAE	encoder gives a cloud (centre + spread); loss = rebuild + stay close to 0 → map without gaps; blurry
GAN	generator (forger) vs discriminator (police), trained in turns; sharp, but unstable, mode collapse
diffusion	learn to remove noise step by step; start from pure noise; Stable Diffusion
in medicine	rare classes, privacy, image-to-image - but never on the test set; check for invented details

Wrap-up

15:45-16:00

Summary

- **autoencoder**: compress → rebuild; the picture is its own label; the latent space is a map that sorts the data - **without labels**
- **denoising AE**: noisy in, clean out · **anomaly detection**: train on normal data, large error = alarm
- **VAE**: cloud instead of point → a map without gaps → new samples (blurry)
- **GAN**: forger vs police → sharp samples, but unstable; **diffusion**: denoise step by step
- **transfer learning** (recap): a pre-trained backbone + a new head - also what Teachable Machine did on day 1